LOGS CENTRALIZADOS Y ESTRUCTURADOS EN SPRING BOOT

```
39:56,001] DEBUG [views.py:15]
39:56,001] INFO [views.py:16]
39:56,001] WARNING [views.py:1
39:56,005] DEBUG [views.py:15]
39:56,005] INFO [views.py:16]
39:56,005] WARNING [views.py:1
39:56,005] ERROR [views.py:18]
39:56,005] CRITICAL [views.py:
39:56,005] CRITICAL [views.py:
39:56,002] ERROR [views.py:18]
39:56,012] CRITICAL [views.py:
39:56,012] CRITICAL [views.py:
03:39,731] CRITICAL [views.py:
03:39,732] CRITICAL [views.py:
```

¿POR QUÉ LA IMPORTANCIA DE LOS LOGS?

- Facilitan la depuración
- Son clave para la detección y solución de errores.
- Proveen visibilidad del sistema en producción
- Ayudan en auditorías y cumplimiento.
- Son esenciales en entornos distribuidos (microservicios).

LOGS ESTRUCTURADOS EN SPRING BOOT

- SLF4J: API de logging común en Java.
- Logback: Implementación por defecto en Spring Boot.
- Configuración en application.yml o logback-spring.xml.
- Permite definir niveles (INFO, DEBUG, ERROR) y salidas (consola, archivo).

JSON LOGGING ESTRUCTURADO

Podemos implementarlo con Logback

```xml
<dependency>
    <groupId>net.logstash.logback</groupId>
    <artifactId>logstash-logback-encoder</artifactId>
    <version>7.4</version>
</dependency>
```

logback-spring.xml

```xml
<configuration>
  <appender name="JSON" class="net.logstash.logback.appender.LoggingEventCompositeJsonEncoder">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder" />
    <file>logs/app.json</file>
  </appender>

  <root level="INFO">
    <appender-ref ref="JSON"/>
  </root>
</configuration>
```

JSON LOGGING ESTRUCTURADO

Structured logging is a technique where the log output is written in a well-defined, often machine-readable format. Spring Boot supports structured logging and has support for the following JSON formats out of the box:

- Elastic Common Schema (ECS)
- Graylog Extended Log Format (GELF)
- Logstash

https://docs.spring.io/spring-boot/reference/features/logging.html#features.logging.structured

application.properties

```
logging.structured.format.file=ecs
logging.file.name=log.json
```

JSON

```
{
  "timestamp": "...",
  "level": "INFO",
  "message": "Servicio iniciado",
  "service": "user-service"
}
```

MDC (MAPPED DIAGNOSTIC CONTEXT) PARA TRAZABILIDAD.

El contexto de diagnóstico mapeado proporciona una manera de enriquecer los mensajes de registro con información que podría no estar disponible en el ámbito donde realmente se produce el registro, pero que puede ser útil para rastrear mejor la ejecución del programa.

¿Por qué lo usamos?

MDC (MAPPED DIAGNOSTIC CONTEXT) PARA TRAZABILIDAD.

1. MDC en Log4j
2. MDC en Log4j2
3. MDC en SLF4J/Logback
4. MDC y Thread Pools

Característica	Log4j 1.x	Log4j2	SLF4J/Logback
API	MDC	ThreadContext	MDC (SLF4J)
Propagación automática	No	No	No
Integración con SLF4J	❌	Necesita puente	✅

INTEGRACIÓN CON LOKI

Grafana Labs desarrolló Loki , un sistema de agregación de registros de código abierto inspirado en Prometheus . Su propósito es almacenar e indexar datos de registros, facilitando la consulta y el análisis eficientes de los registros generados por diversas aplicaciones y sistemas.

¿Cómo funciona esta integración?

- La aplicación genera logs estructurados en JSON.
- Un agente de logs (como Promtail o Fluent Bit) recolecta estos logs.
- El agente envía los logs a Loki.
- Loki almacena los logs y los pone a disposición para ser consultados en Grafana.

BENEFICIOS

- ✓ Trazabilidad Mejorada con MDC
- ✓ Más fácil de debuggear por logs de JSON estructurados.
- ✓ Mejor monitoreo con Loki + Grafana
- ✓ Auditoría y cumplimiento gracias a logs centralizados
- ✓ Resolución de problemas más rápida